

Jenkins CI: Test to Image

Reproducible steps • Test gates • Understand exactly what each artifact contains

SAY IT IN THE INTERVIEW

“My Jenkins pipeline checks out main, runs each service's tests, packages and archives its JAR, then builds both Docker images. Test stages run before packaging. I use skipTests during packaging to avoid running the same tests twice, not to bypass the earlier test gates.”

01 THE ACTUAL STAGE ORDER

- **Checkout main** → Order test → Order package → Order JAR archive → Inventory test → Inventory package → Inventory JAR archive → build both Docker images.
- These stages are sequential in the current Jenkinsfile. A failed required test command stops later normal stages; image publication follows the successful CI path.
- **Continuous integration:** automatically build and verify integrated changes. CD and the manual deployment decision are covered in Chapter 13.

02 TEST ENVIRONMENT MATTERS

- Order: `./mvnw test` with a datasource URL override to host MySQL **order_db**. This is not a dedicated Order test database.
- Inventory: `./mvnw test` with **TEST_DB_URL** pointing to **inventory_test_db** and **TEST_SCHEMA_REGISTRY_URL** pointing to host port **8081**.
- Embedded Kafka supplies the broker for the DLT integration test; MySQL and Schema Registry remain external dependencies. “Embedded” does not mean the whole test environment is self-contained.

03 JAR AND IMAGE ARE DIFFERENT OUTPUTS

- `./mvnw package -DskipTests` packages each service. Jenkins archives **service/target/*.jar** for download from the build.
- docker build uses each service directory and Dockerfile. Local tags are **order-service:\${BUILD_NUMBER}** and **inventory-service:\${BUILD_NUMBER}**.
- A JAR archive does not publish a Docker image. An image build does not push to a registry. Push stages are separate (Chapter 12).

04 WHEN A BUILD FAILS, WHAT DO I CHECK?

- First identify the failing stage. For tests, inspect the Maven failure and DB/Registry reachability. For image builds, check that packaging produced the expected JAR.
- Use the checked-out **Git SHA + Jenkins build number** to identify the source and build. A moving main branch name alone is insufficient.
- Current Jenkinsfile archives JARs but does not publish JUnit XML through a junit step. The tests exist; a coverage percentage is not measured by counting test methods.

YAAD RAKHO: test gate → package JAR → archive JAR → build image → push later.

SOURCE CHECK: Jenkinsfile test/package/archive/build stages + service test configurations